Decision Trees

Methodology:

Before constructing the tree, I called a method to identify the types of the various columns(categorical/numerical). This pre-processing was useful for deciding whether to split the data into two or more categories.I constructed the tree in a recursive fashion. I started at the root and recursively made splits on the data at the root by choosing to split on the feature which gave the maximum information gain.

The entropy is given by,

$$\mathrm{H}(T) = \mathrm{I}_E(p_1, p_2, \ldots, p_J) = -\sum_{i=1}^{J} p_i \log_2 p_i$$

The mutual information is given by,

$$\overbrace{IG(T,a)}^{\text{information gain}} = \overbrace{\mathrm{H}(T)}^{\text{entropy (parent)}} - \overbrace{\mathrm{H}(T \mid a)}^{\text{sum of entropies (children)}}$$

$$= -\sum_{i=1}^{J} p_i \log_2 p_i - \sum_{i=1}^{J} -\Pr(i \mid a) \log_2 \Pr(i \mid a)$$

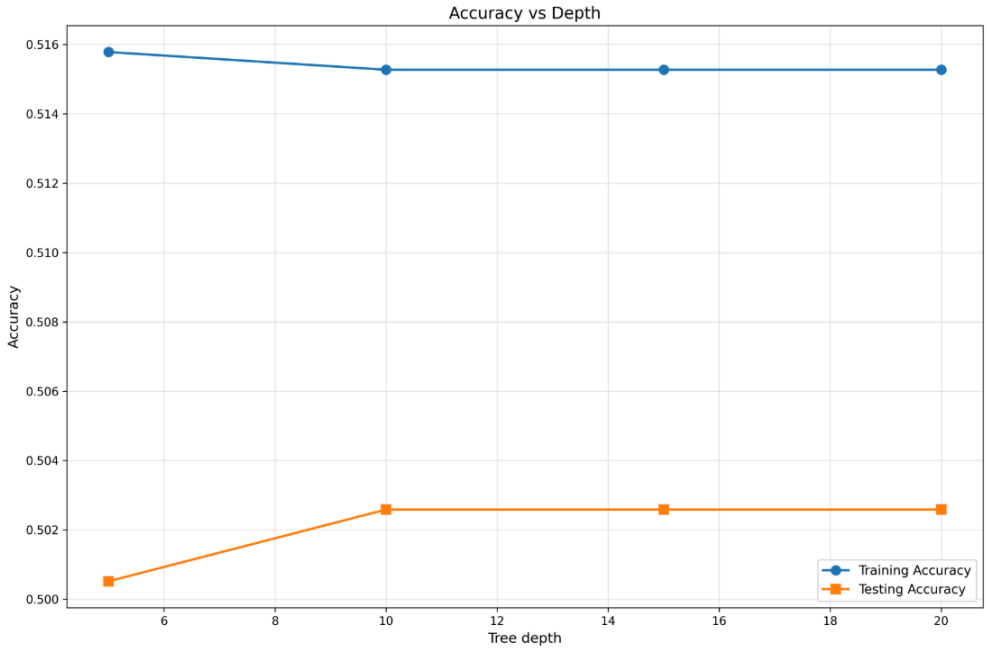
Gini impurity is given by,

$$\mathrm{I}_G(p) = \sum_{i=1}^{J} \left(p_i \sum_{k \neq i} p_k \right) = \sum_{i=1}^{J} p_i(1 - p_i) = \sum_{i=1}^{J} (p_i - p_i^2) = \sum_{i=1}^{J} p_i - \sum_{i=1}^{J} p_i^2 = 1 - \sum_{i=1}^{J} p_i^2.$$

I stopped splitting in case there were no more samples or the max depth had been reached. While predicting the value for a certain entry, I used dfs to traverse through the tree. Which node to move to next was determined by the value of the entry corresponding to the feature on which it was split on at that node. The traversal is continued till a leaf is reached and the majority class at the leaf is reported. Post pruning is done greedily by removing(pruning) the node whose removal gives maximum accuracy gain on the validation set.

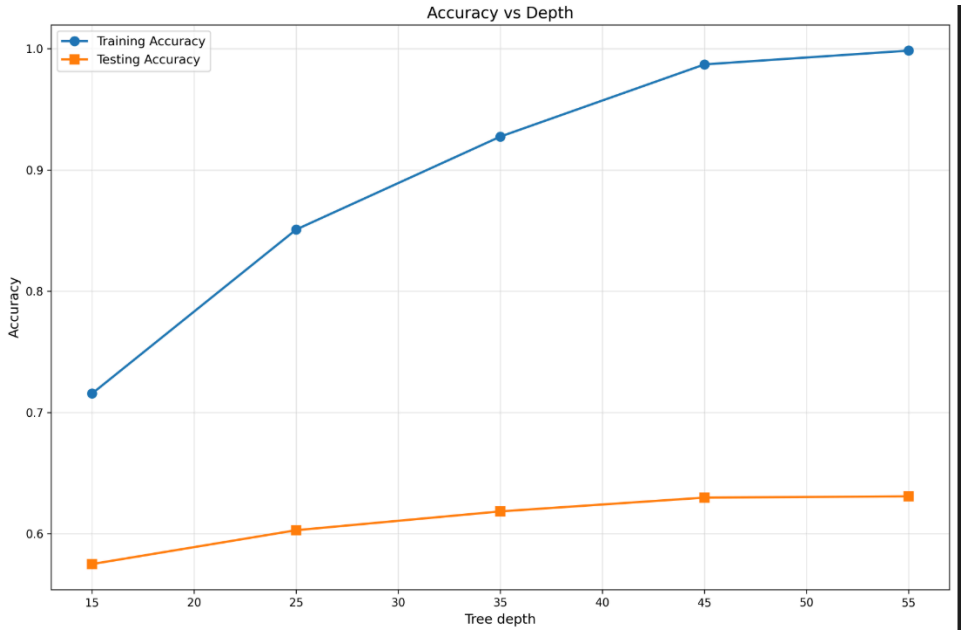
Evaluation metrics

Part a)



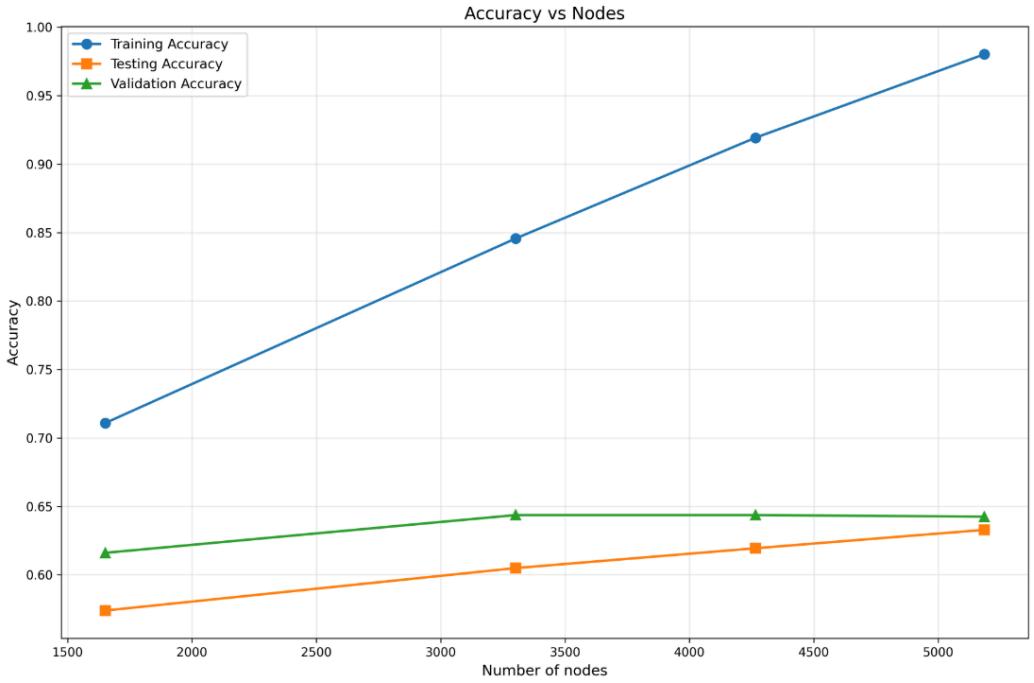
We observe that both the training accuracy and testing accuracy are around 50%. It is important to keep in mind that the probability when randomly guessing is 50% (assuming that the data is evenly distributed). Since both the accuracies are near to the baseline, the model is severely underfitting on the data

Part b)



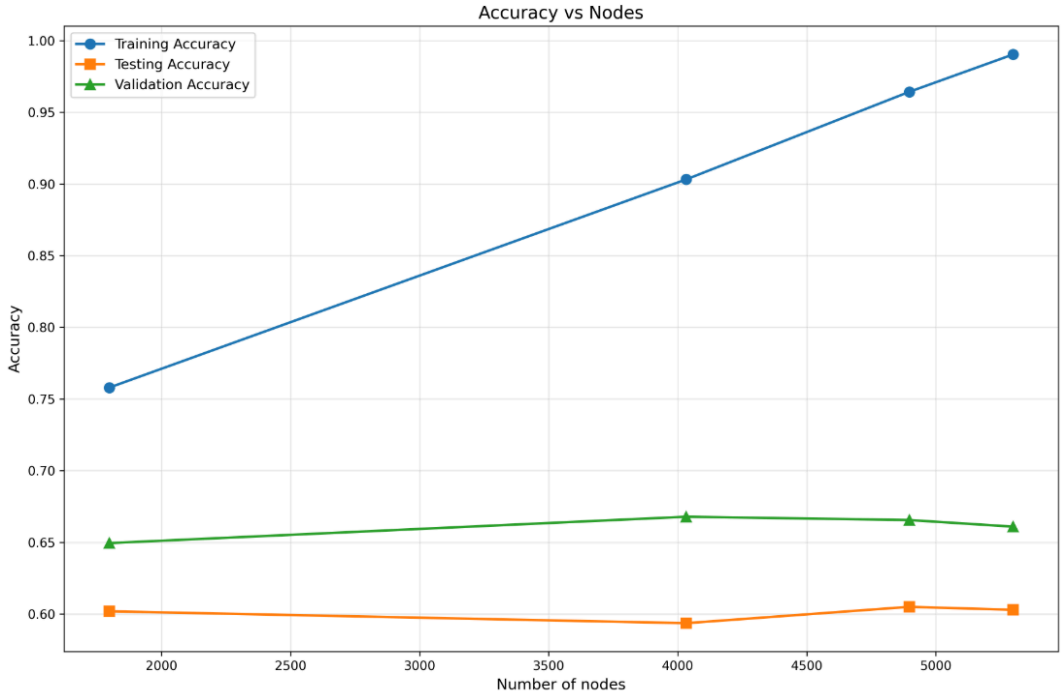
After doing the encoding, we observe that the testing and training accuracies are increasing with increase in depth. The testing accuracies settle around 62% which is an improvement over 50%. No significant improvement in testing accuracy is observed upon increasing the depth beyond 55.

Part c)



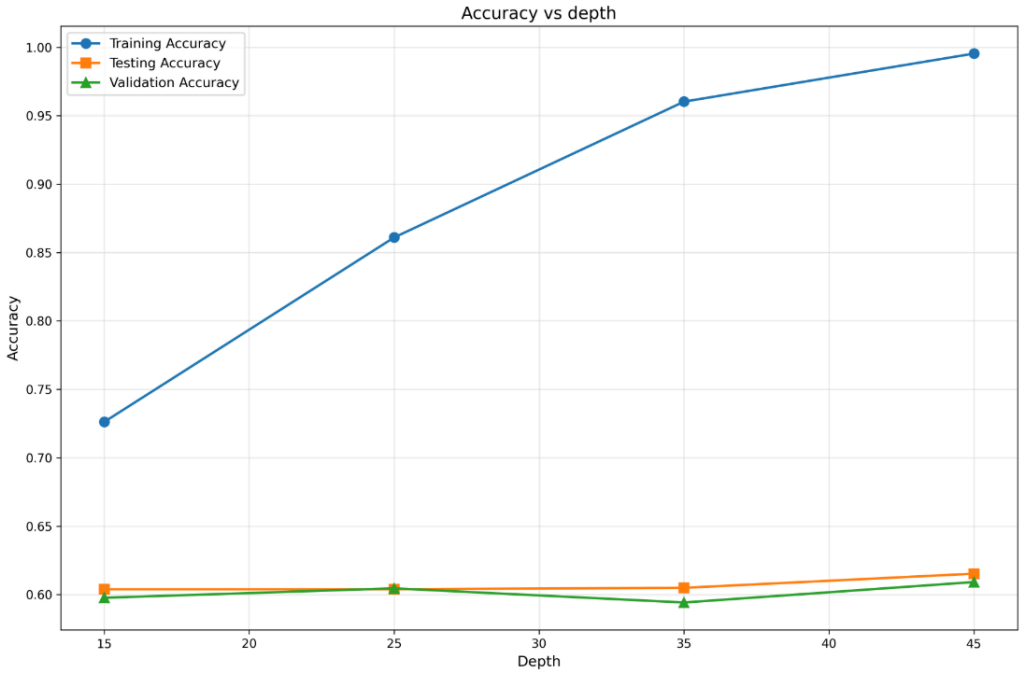
After pruning on the dataset from part b) we observe that the testing accuracy increases to 64% using max_depth as 45. We observe that all the models after max_depth 15 settle to the same validation accuracy

Part d)

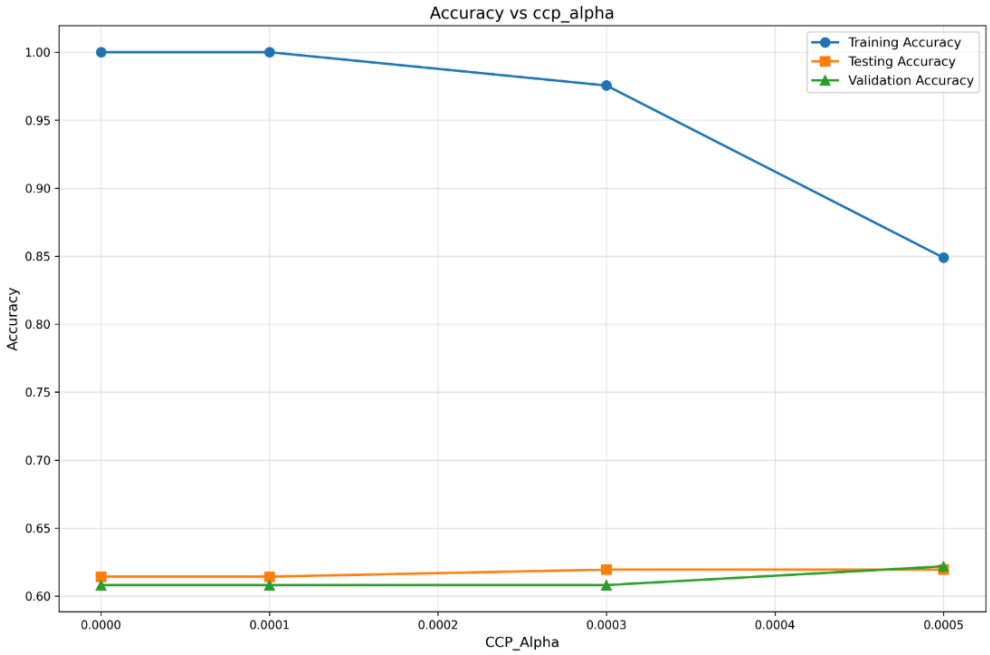


We observe that for our dataset, setting the criterion to ‘gini impurity’ leads to less testing accuracy as compared to ‘entropy’. The testing accuracy is around 60%

Part e)



We get the above graph by varying depth. We observe that test accuracy is around 62% like in part b). Also the depth with max test accuracy is 45 like in part b). Best max_depth=45



We get the above graph by varying the pruning parameter (ccp_alpha). As the value of the pruning parameter increases, we observe a slight increase in the value of test accuracy to 62%. This is similar to part c) but part c) and this cannot by directly compared as the max_depth was varying in case of part c)
Best pruning parameter: ccp_alpha=0.0005

Part f)
BEST MODEL (Based on Validation Accuracy)
n_estimators: 50 ,max_features: 0.7 ,min_samples_split: 8

Training Accuracy: 0.9856, Out-of-Bag Accuracy: 0.6995
Validation Accuracy: 0.7195, Test Accuracy: 0.7022

Confusion Matrix (Test Set):
[[358 151]
[137 321]]

We see that the accuracy has further improved to 70% likely due to the fact that more richer information can be captured by as ensemble of decision trees as compared to one

Neural_Networks

Methodology:

The hidden layers and their corresponding biases and weights are initialized according to the hidden layer architecture mentioned. The input layer is decided based on the input and the output layer is decided based on the number of target classes. The activation layer is applied to the linear combination of input(output from previous layer) and the weights added with the bias. This then becomes the input to the next layer. This process is called forward pass. This process is repeated till we get to the output layer where it is passed through a softmax layer to get the probabilities of each class. We get the loss by using the cross-entropy between these probabilities and the one-hot encoded values of the training target value. This loss is then backpropagated to update the weights and biases. This process of forward and backward propagation occurs in a single epoch but splitting the input into various batches and applying stochastic gradient descent

Forward propagation

For a network with L layers:

$$\text{For each layer } l = 1, 2, \ldots, L:$$

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = \begin{cases} f(z^{[l]}) & \text{for hidden layers} \\ \text{softmax}(z^{[l]}) & \text{for output layer} \end{cases}$$

where:

- $a^{[0]} = X$ (input)
- $W^{[l]}$ = weight matrix of layer l
- $b^{[l]}$ = bias vector of layer l
- $f(\cdot)$ = activation function

🔁 4. Backward Propagation

Output Layer Delta

$$\delta^{[L]} = Y - A^{[L]}$$

(where $A^{[L]} = \hat{Y}$)

Hidden Layer Delta

$$\delta^{[l]} = (\delta^{[l+1]} W^{[l+1]}) \odot f'(A^{[l]})$$

where:

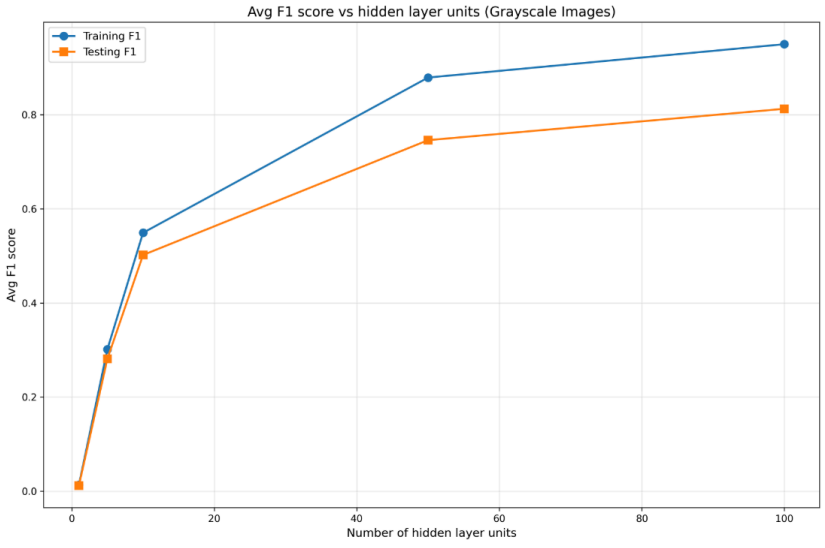
- $\odot$ = element-wise multiplication
- $f'(A^{[l]})$ = derivative of activation for layer l

Evaluation metrics:

Part b)

Num of Hidden Units | Train Acc | Test Acc | Train F1 | Test F1

1	0.0541	0.0526	0.0131	0.0118
5	0.3423	0.3206	0.3016	0.2812
10	0.5587	0.5115	0.5491	0.5019
50	0.8790	0.7458	0.8791	0.7460
100	0.9501	0.8133	0.9500	0.8127



We observe that the training and testing accuracy increase and the testing accuracy eventually settles to around 80%

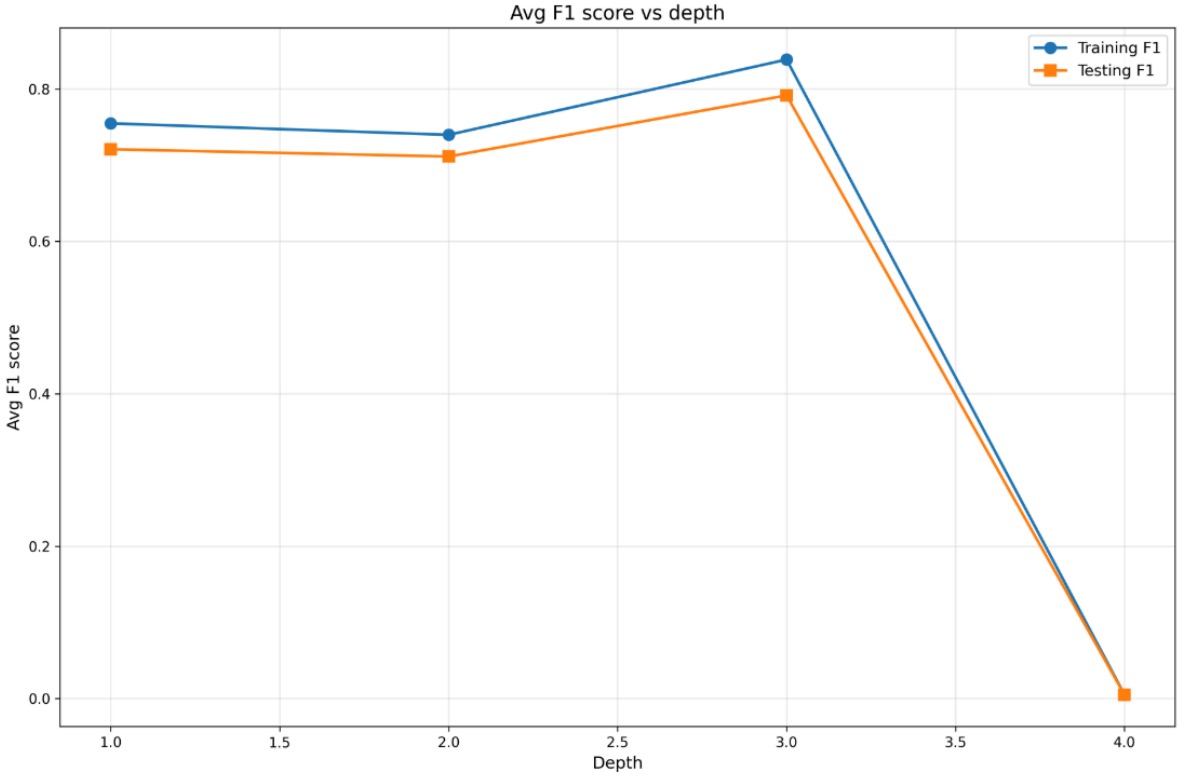
Part c)

Part c

Hidden Units | Train Acc | Test Acc | Train F1 | Test F1

---

1 | 0.7559 | 0.7219 | 0.7550 | 0.7210
2 | 0.7413 | 0.7129 | 0.7399 | 0.7115
3 | 0.8390 | 0.7919 | 0.8387 | 0.7917
4 | 0.0460 | 0.0461 | 0.0049 | 0.0049



We observe that although both the accuracies increase till the layer depth is 3, for depth 4 both accuracies go to 0. When using sigmoid activation functions throughout the network, the vanishing gradient problem causes training and test accuracy to drop to zero due to the multiplicative nature of backpropagation through the chain rule. Since the derivative of the sigmoid function is bounded by a maximum value of 0.25, gradients in deep networks are computed as the product of these small derivatives across multiple layers, resulting in exponentially decaying gradients - for example, in a 4-layer network, gradients in early layers become approximately $(0.25)^3 \approx 0.016$ times smaller than in the

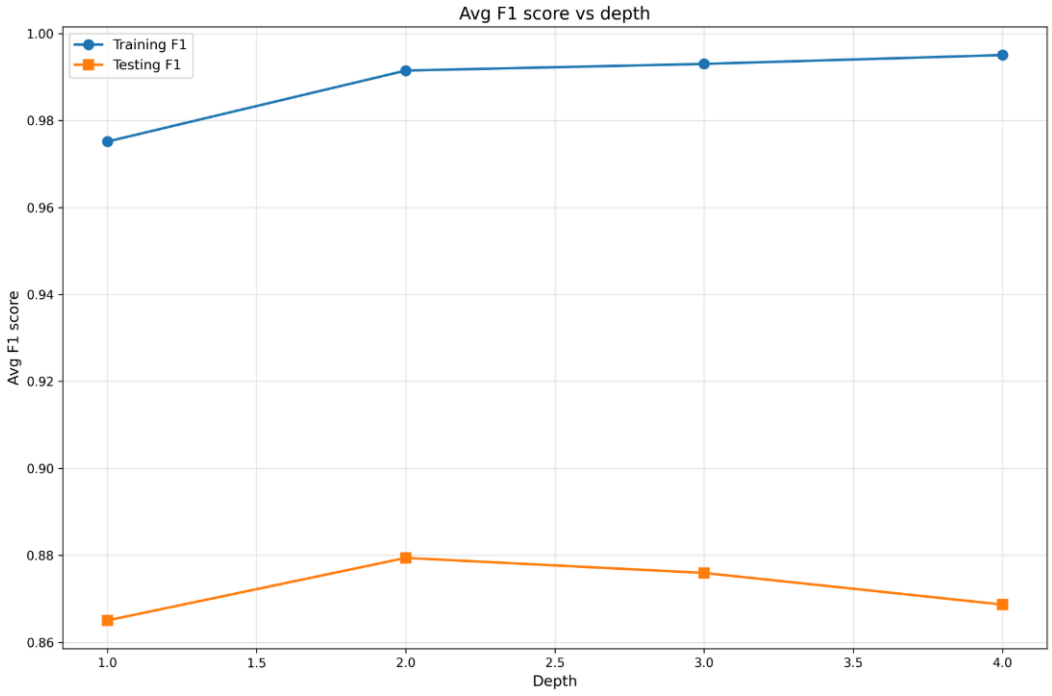
output layer. This exponential decay effectively prevents early layers from learning any meaningful features, leaving the network stuck at or near its random initialization. Furthermore, sigmoid functions saturate when their inputs are large in magnitude ($|z| > 5$), producing outputs near 0 or 1 with derivatives approaching zero, which completely halts gradient flow and causes all neurons to output nearly identical values. Consequently, the network degenerates into predicting only a single class for all inputs, and if the test set is imbalanced or uses different label encoding than the predominant predicted class, the accuracy becomes zero. This problem is mitigated by using ReLU activation functions in hidden layers, which have a constant derivative of 1 for positive inputs, preventing gradient vanishing while maintaining the ability to learn complex features across deep architectures.

Part d)

part d

Hidden Units | Train Acc | Test Acc | Train F1 | Test F1

1	0.9751	0.8649	0.9752	0.8650
2	0.9915	0.8795	0.9915	0.8794
3	0.9930	0.8756	0.9930	0.8760
4	0.9950	0.8683	0.9950	0.8687



We observe that the training accuracy is better than that of part c). Vanishing gradients do not occur with ReLU activation because its derivative is exactly 1 for positive inputs (rather than sigmoid's maximum of 0.25). During backpropagation through an L-layer network, gradients multiply by activation derivatives at each layer - with sigmoid this yields gradients proportional to (0.25)^L causing exponential decay, whereas ReLU maintains gradients at 1^L = 1 when neurons are active, allowing gradients to flow unchanged through multiple layers. This enables early layers to receive gradients of similar magnitude to later layers and learn effectively, preventing the accuracy drop to zero that occurs with sigmoid networks at increased depth. However, the testing accuracy starts to decrease due to overfitting as the network becomes deeper.

Part e)

Part e

Depth | Architecture | Train Acc | Test Acc | Train F1 | Test F1

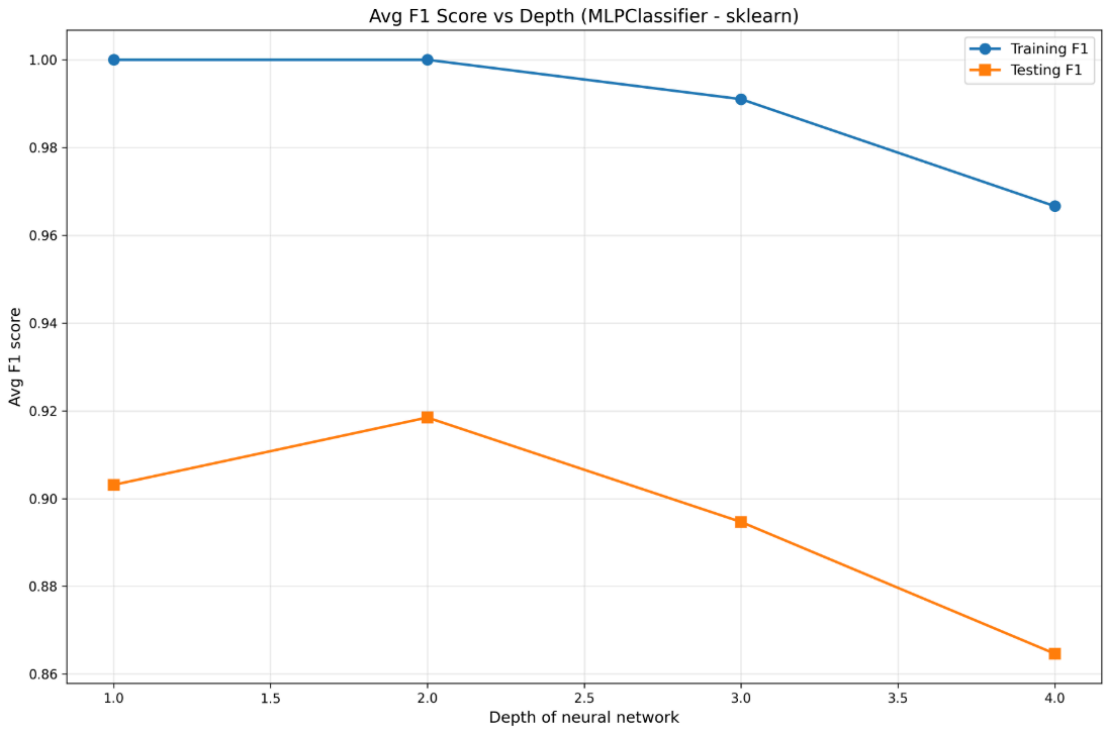
---

1 | (512,) | 1.0000 | 0.9032 | 1.0000 | 0.9031

2 | (512, 256) | 1.0000 | 0.9185 | 1.0000 | 0.9185

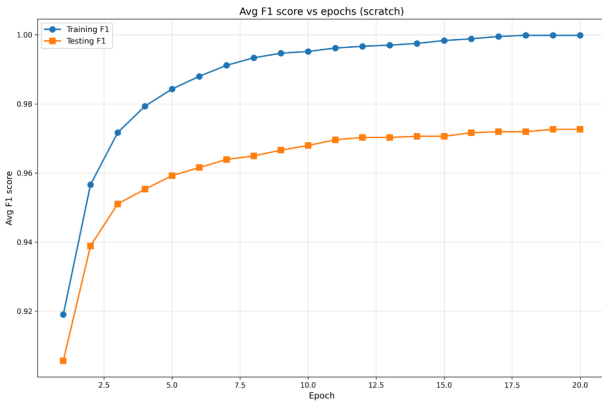
3 | (512, 256, 128) | 0.9910 | 0.8943 | 0.9910 | 0.8947

4 | (512, 256, 128, 64) | 0.9665 | 0.8626 | 0.9666 | 0.8647

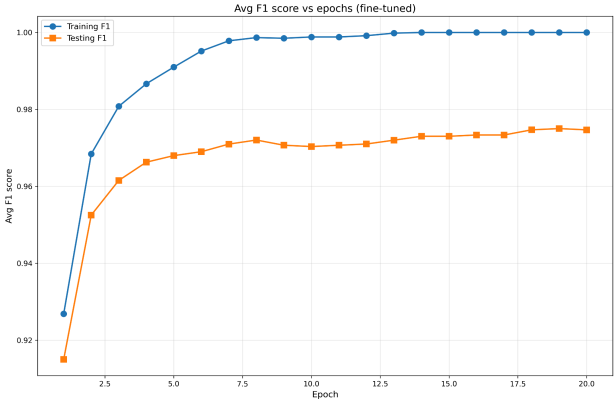


The results are comparable to that obtained in part d). The model starts to overfit as we start to increase the network depth beyond a limit. The accuracies are also comparable.

Part f)



Training from scratch



Training using fine-tuned model

We observe that the final testing accuracy using the fine-tuned model is comparable to the model trained from scratch. Moreover, the slope of the graph on the right is greater initially. This indicates that the settling value can be obtained in fewer epochs. So the training time for the model on the right is less leading to greater efficiency